QRM — Quality Requirements Module

QRM — Quality Requirements Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Quality Standard (SQS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Quality Requirements Governance

Module ID: SLGA-SQS-QRM-001

Origin ID: OOF.CM.SLGA.05.01.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Quality Requirements Module (QRM) governs the identification,

establishment, classification, contextualization, authorization, consistency, traceability, and lifecycle continuity of software quality requirements.

QRM establishes the methodological conditions under which a software

quality expectation can be recognized as an identifiable, justified, sufficiently specified, contextually applicable, and legitimately established requirement for subsequent software quality assessment.

It governs the relationship between software purpose, intended use,

relevant stakeholders, applicable software characteristics, quality expectations, measurable or otherwise assessable quality conditions, acceptance expectations, operational constraints, and the authority under which quality requirements are established or changed.

QRM distinguishes stakeholder expectations from governed quality

requirements, quality objectives from assessable obligations, declared quality thresholds from justified thresholds, and requirements establishment from demonstrated software quality.

It applies to software developed, modified, maintained, assessed, or

operated by humans, organizations, automated software systems, AI coding assistants, and autonomous development agents.

QRM is technology-independent and does not prescribe a particular programming language, testing framework, software development method, or quality management platform.

Its principal output is:

Governed Software Quality Requirements State

Its foundational distinction is:

Quality Expectation ≠ Governed Quality Requirement.

Operational Role

The operational role of QRM is:

To ensure that software quality expectations are transformed into identifiable, justified, authorized, contextually applicable, sufficiently specified, and traceable quality requirements before they are used as the basis for software quality criteria, testing, validation, or acceptance.

QRM establishes the requirements-governance foundation of SQS.

It identifies which software quality conditions matter, why they matter, to

which software objects and operating contexts they apply, and under whose authority they become recognized requirements.

QRM does not independently demonstrate that the software satisfies those requirements.

It provides the governed requirements state required by the subsequent SQS

modules.

Operational Question

Which software quality conditions must be satisfied, for which software object and intended use, under whose authority, within which operational boundaries, and on what justified basis can those conditions be recognized as governed quality requirements?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

QRM operates within:

Software Quality Standard (SQS)

The SQS internal governance flow is:

Quality Requirements → Quality Criteria Definition → Testing & Coverage

Management → Defect Management → Quality Conformity & Acceptance

QRM governs:

Quality Requirements

It is the first Core Module of SQS.

Its principal upstream references include:

Governed Software Requirements Baseline

from:

Software Requirements Standard (SRS)

Governed Software Creation State

from:

Software Creation Standard (SCS)

Governed Software Dependency State

from:

Software Dependency Standard (SDS)

Governed Software Configuration State

from:

Software Configuration Standard (SCFS)

Its principal output is:

Governed Software Quality Requirements State

This output supports:

QCDM — Quality Criteria Definition Module

and contributes to:

Governed Software Quality State

QRM may be invoked whenever software quality requirements are initially established, revised, challenged, superseded, or reassessed.

• Governed Space

Software Quality Requirements Governance

This space governs the conditions under which software quality expectations

become sufficiently established requirements.

Its principal dimensions include:

• Quality requirement governance object identity

• Quality requirement purpose

• Software object correspondence

• Intended-use correspondence

• Stakeholder quality expectations

• Quality requirement origin

• Quality requirement justification

• Quality requirement authority

• Quality requirement scope

• Quality characteristic identification

• Quality requirement classification

• Functional quality relationships

• Non-functional quality relationships

• Reliability requirements

• Performance requirements

• Compatibility requirements

• Usability requirements

• Maintainability requirements

• Portability requirements

• Recoverability requirements

• Quality requirement priority

• Quality requirement criticality

• Quality requirement thresholds

• Quality requirement constraints

• Quality requirement assumptions

• Quality requirement dependencies

• Quality requirement consistency

• Quality requirement completeness

• Quality requirement feasibility

• Quality requirement assessability

• Quality requirement traceability

• Quality requirement acceptance expectations

• Quality requirement evidence

• Evidence competence

• Evidence integrity

• Quality requirement conflicts

• Quality requirement uncertainty

• Quality requirement gaps

• Critical quality requirement gaps

• Quality requirement change

• Quality requirement supersession

• Quality requirement continuity

• Quality requirement reassessment

• Quality requirement governance states

This governed space remains distinct from general software requirements

governance under SRS and from detailed quality criteria definition under QCDM.

Quality Requirement Governance Object

Every material QRM assessment shall establish a Quality Requirement

Governance Object.

The object represents an identifiable software quality requirement and its

governance conditions.

It should identify:

Requirement Identity: The identifiable quality requirement.

Requirement Purpose: The reason the requirement exists.

Software Object: The software to which it applies.

Intended Use: The relevant use and operating context.

Requirement Origin: The source of the expectation.

Requirement Authority: The legitimate basis for establishment.

Quality Characteristic: The relevant quality dimension.

Requirement Statement: The expected quality condition.

Requirement Scope: The applicable boundaries.

Requirement Constraints: Material restrictions.

Requirement Dependencies: Relevant prerequisite relationships.

Requirement Priority: The recognized importance.

Requirement Criticality: The material consequence of non-satisfaction.

Requirement Evidence: Supporting justification and establishment evidence.

Requirement State: The recognized governance condition.

The governance object shall remain distinguishable from a stakeholder

comment, feature request, development ticket, or quality metric.

Quality Requirement Governance Object ≠ Stakeholder Quality Request.

Quality Requirement Purpose Governance

QRM shall identify the purpose of material software quality requirements.

Purposes may include:

• Ensuring functional suitability

• Establishing reliability expectations

• Defining performance conditions

• Supporting compatibility

• Establishing usability expectations

• Supporting maintainability

• Preserving portability

• Establishing recoverability conditions

• Supporting lifecycle continuity

Purpose shall remain connected to the relevant software object and intended

use.

Quality Requirement Purpose ≠ Quality Requirement Satisfaction.

Software Object Correspondence

QRM shall identify the software object to which a quality requirement

applies.

Objects may include:

• Software component

• Application

• Service

• API

• Library

• Embedded software

• Integrated software system

• Specific software version

• Defined software configuration

Requirements shall remain associated with sufficiently identifiable

software objects.

Quality Requirement for One Software Object ≠ Quality Requirement for

Every Related Software Object.

Intended-Use Correspondence

QRM shall establish the intended-use context relevant to the quality

requirement.

Context may include:

• User tasks

• Operational purpose

• Expected workloads

• Relevant environments

• Integration conditions

• Data characteristics

• Service expectations

• Relevant failure consequences

A quality requirement shall not be generalized beyond its applicable

intended-use context without competent justification.

Quality Expectation in One Context ≠ Universal Quality Requirement.

Stakeholder Quality Expectation Governance

QRM shall recognize quality expectations originating from relevant

stakeholders.

Stakeholders may include:

• Software users

• Product owners

• Developers

• Operators

• Customers

• Infrastructure owners

• Quality specialists

• Maintenance teams

• Relevant institutional authorities

Stakeholder expectations shall remain distinguishable from legitimately

established requirements.

Stakeholder Quality Expectation ≠ Authorized Quality Requirement.

Quality Requirement Origin

QRM shall identify the origin of material quality requirements.

Origins may include:

• Governed software requirements

• Intended-use needs

• Operational experience

• Contractual obligations

• Applicable technical specifications

• Maintenance needs

• Relevant quality findings

• Competent organizational decisions

Origin shall remain distinguishable from requirement authority.

Requirement Origin ≠ Requirement Authorization.

Quality Requirement Justification

QRM shall preserve the reason supporting a material quality requirement.

Justification may concern:

• User need

• Operational reliability

• Functional correctness

• Performance demand

• Compatibility

• Maintenance efficiency

• Recovery expectations

• Known failure consequences

A plausible justification shall not independently establish a governed

requirement.

Quality Requirement Justification ≠ Quality Requirement Approval.

Quality Requirement Authority

QRM shall identify the legitimate basis for establishing material quality

requirements.

Authority may arise through:

• Competent organizational approval

• Authorized product responsibility

• Contractual conditions

• Applicable institutional requirements

• Delegated requirements governance

• Approved software governance procedures

Authority shall remain bounded by the relevant software object and scope.

Ability to Propose a Quality Requirement ≠ Authority to Establish It.

Quality Requirement Scope

QRM shall establish the boundaries within which a quality requirement applies.

Scope may concern:

• Software object

• Function

• Component

• User group

• Environment

• Workload

• Operating condition

• Lifecycle stage

• Applicable version

Scope shall distinguish included, excluded, and unresolved conditions.

Defined Requirement Scope ≠ Universal Requirement Applicability.

Quality Characteristic Identification

QRM shall identify the software quality characteristic governed by each

material requirement.

Characteristics may include:

• Functional suitability

• Reliability

• Performance efficiency

• Compatibility

• Usability

• Maintainability

• Portability

• Recoverability

Quality characteristics shall be selected according to the actual software

purpose and relevant governance context.

Quality Characteristic Identification ≠ Quality Characteristic

Demonstration.

Quality Requirement Classification

QRM shall classify material software quality requirements according to

their governance purpose and characteristics.

Classifications may include:

• Functional quality requirement

• Reliability requirement

• Performance requirement

• Compatibility requirement

• Usability requirement

• Maintainability requirement

• Portability requirement

• Recoverability requirement

• Cross-characteristic requirement

Classification shall remain distinguishable from priority or criticality.

Quality Requirement Classification ≠ Quality Requirement Criticality.

Functional Quality Requirements

QRM shall recognize requirements concerning the suitability and correctness

of defined software functions.

Such requirements may address:

• Required functional outcomes

• Input processing

• Output correctness

• Interface behavior

• Error handling

• Relevant boundary conditions

Functional quality requirements shall remain traceable to the applicable

governed software requirements.

Defined Functional Quality Requirement ≠ Verified Functional Correctness.

Reliability Requirements

QRM shall establish relevant reliability expectations.

Requirements may concern:

• Failure frequency

• Operational stability

• Fault tolerance

• Availability

• Recovery behavior

• Sustained operation

• Relevant failure conditions

Reliability requirements shall remain contextualized to intended operating

conditions.

Reliability Requirement ≠ Demonstrated Reliability.

Performance Requirements

QRM shall establish relevant software performance expectations.

Requirements may concern:

• Response time

• Throughput

• Resource efficiency

• Scalability

• Workload capacity

• Performance degradation

Performance expectations shall preserve their workload and environment

context.

Performance Target ≠ Demonstrated Performance.

Compatibility Requirements

QRM shall establish relevant software compatibility expectations.

Requirements may concern:

• Operating systems

• Hardware platforms

• Software versions

• Interfaces

• Data formats

• Integration environments

Compatibility requirements shall identify the combinations to which they

apply.

Compatibility Requirement for One Platform ≠ Universal Compatibility

Requirement.

Usability Requirements

QRM shall recognize relevant usability and interaction quality expectations.

Requirements may concern:

• Task completion

• Interaction clarity

• Accessibility

• Error prevention

• Interaction consistency

• Recovery from user errors

Usability requirements shall remain associated with relevant users, tasks,

and contexts.

Usability Expectation ≠ Demonstrated Usability.

Maintainability Requirements

QRM shall establish relevant software maintainability expectations.

Requirements may concern:

• Modularity

• Analyzability

• Changeability

• Testability

• Complexity

• Documentation correspondence

• Maintenance constraints

Maintainability requirements shall remain assessable within the relevant software context.

Maintainability Requirement ≠ Demonstrated Maintainability.

Portability Requirements

QRM shall establish relevant software portability and adaptability

expectations.

Requirements may concern:

• Installation

• Platform migration

• Environment adaptation

• Configuration flexibility

• Supported target environments

Portability requirements shall preserve applicable platform and environment

boundaries.

Portable Design Objective ≠ Verified Portability.

Recoverability Requirements

QRM shall establish relevant software recovery expectations.

Requirements may concern:

• Recovery time

• Restoration conditions

• Recovery behavior

• State restoration

• Error recovery

• Relevant continuity conditions

Recoverability requirements shall remain distinguishable from demonstrated

recovery capability.

Recovery Requirement ≠ Verified Recovery Capability.

Quality Requirement Priority

QRM shall identify the recognized priority of material quality requirements.

Priority may reflect:

• Delivery needs

• Product objectives

• Operational dependencies

• Resource constraints

• Stakeholder importance

• Lifecycle timing

Priority shall remain distinguishable from the consequence of

non-satisfaction.

Requirement Priority ≠ Requirement Criticality.

Quality Requirement Criticality

QRM shall assess the material significance of failing to satisfy a quality

requirement.

Criticality may depend on:

• Functional consequences

• Operational disruption

• Affected users

• Recovery limitations

• Dependency relationships

• Material service impacts

Criticality shall remain evidence-supported and context-dependent.

High Requirement Priority ≠ High Requirement Criticality.

Quality Requirement Thresholds

QRM shall recognize material quantitative or qualitative thresholds

associated with quality requirements.

Thresholds may include:

• Maximum response time

• Minimum throughput

• Defined availability

• Maximum acceptable error frequency

• Required compatibility conditions

• Recovery expectations

QRM establishes the requirement-level threshold.

QCDM governs its detailed assessment criteria and evidence conditions.

Declared Quality Threshold ≠ Justified Quality Threshold.

Quality Requirement Constraints

QRM shall identify material constraints affecting quality requirements.

Constraints may concern:

• Technical limitations

• Operating environments

• Resource availability

• Platform conditions

• Dependency restrictions

• Relevant organizational obligations

• Lifecycle constraints

Constraints shall remain distinguishable from requirements themselves.

Quality Requirement Constraint ≠ Quality Requirement Satisfaction.

Quality Requirement Assumptions

QRM shall preserve material assumptions underlying quality requirements.

Assumptions may concern:

• Expected workload

• User behavior

• Operating environment

• Data characteristics

• Infrastructure availability

• External service behavior

Where assumptions are not sufficiently supported, uncertainty shall remain explicit.

Quality Requirement Assumption ≠ Verified Operating Condition.

Quality Requirement Dependencies

QRM shall identify material relationships between quality requirements and

prerequisite conditions.

Dependencies may concern:

• Software functionality

• Infrastructure

• External services

• Configuration

• Component behavior

• Data availability

• Other quality requirements

Dependency identification shall not independently establish that prerequisite conditions are satisfied.

Quality Requirement Dependency Identified ≠ Dependency Condition

Satisfied.

Quality Requirement Consistency

QRM shall assess whether material quality requirements are mutually consistent.

Conflicts may include:

• Incompatible performance and resource constraints

• Contradictory availability expectations

• Conflicting platform requirements

• Incompatible usability and operational constraints

• Contradictory acceptance expectations

Consistency shall be evaluated within the relevant software context.

Individually Valid Requirements ≠ Collectively Consistent Requirements.

Quality Requirement Completeness

QRM shall assess whether material quality expectations have been

sufficiently identified for the defined scope and intended use.

Completeness may concern:

• Relevant quality characteristics

• Intended operating conditions

• Material failure scenarios

• Important user tasks

• Critical dependencies

• Required acceptance conditions

Completeness shall remain bounded by the assessed purpose and available

evidence.

Documented Quality Requirements ≠ Complete Quality Requirements.

Quality Requirement Feasibility

QRM shall assess whether material quality requirements are sufficiently feasible under relevant constraints.

Feasibility may involve:

• Technical capability

• Available resources

• Relevant platform limits

• Operational conditions

• Dependency behavior

• Time constraints

• Measurement capability

Feasibility assessment shall remain distinguishable from satisfaction.

Feasible Quality Requirement ≠ Satisfied Quality Requirement.

Quality Requirement Assessability

QRM shall determine whether a quality requirement can be meaningfully assessed.

Assessability may depend on:

• Identifiable expected condition

• Defined software object

• Relevant context

• Suitable observation method

• Sufficiently specific requirement statement

• Available evidence sources

Detailed criteria design remains governed by QCDM.

Quality Requirement Statement ≠ Assessable Quality Requirement.

Quality Requirement Traceability

QRM shall preserve material relationships between quality requirements and

their origins, purposes, software objects, and downstream assessments.

Traceability may connect:

• Requirement origin

• Requirement authority

• Intended use

• Software object

• Quality characteristic

• Requirement statement

• Applicable constraints

• Downstream quality criteria

Traceability shall not independently establish requirement validity.

Traceable Quality Requirement ≠ Validated Quality Requirement.

Quality Acceptance Expectations

QRM shall identify material expectations concerning quality acceptance.

Expectations may include:

• Required quality characteristics

• Applicable thresholds

• Required evidence categories

• Relevant limitations

• Acceptance authority references

• Conditions requiring reassessment

QRM governs the requirements-level expectations.

QCAM governs actual conformity and acceptance decisions.

Quality Acceptance Expectation ≠ Quality Acceptance Decision.

Quality Requirement Evidence

QRM shall preserve evidence supporting material quality requirement claims.

Evidence may include:

• Stakeholder requirements

• Governed software requirements

• Intended-use documentation

• Operational evidence

• Relevant technical specifications

• Requirement justification

• Approval records

• Historical quality findings

Evidence shall remain associated with the specific requirement claim it

supports.

Quality Requirement Evidence Presence ≠ Requirement Validity.

Evidence Competence

QRM shall assess whether evidence is competent to support the relevant quality requirement determination.

For example:

A stakeholder statement may support the existence of an expectation.

A governed requirements baseline may support requirement origin.

An operational observation may support the need for a reliability requirement.

An approval record may support establishment authority.

These records establish different facts.

Evidence of Quality Need ≠ Evidence of Requirement Authorization.

Evidence Integrity

QRM shall preserve the integrity of material quality requirement evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Software object correspondence

• Requirement identity correspondence

• Material contradictions

• Preservation history

An intact requirement record does not independently establish that the requirement is justified or applicable.

Intact Quality Requirement Record ≠ Governed Quality Requirement.

Quality Requirement Conflict

A Quality Requirement Conflict exists where material requirements or supporting claims are incompatible.

Examples may include:

• Contradictory performance thresholds

• Incompatible reliability conditions

• Conflicting stakeholder expectations

• Contradictory scope statements

• Conflicting approval records

QRM shall preserve the conflict and its supporting evidence.

Conflicting Quality Requirements ≠ Permission to Select the Preferred

Requirement.

Quality Requirement Uncertainty

QRM shall preserve material uncertainty concerning quality requirement

identity, scope, justification, authority, or applicability.

Uncertainty may arise where:

• Intended use is unclear

• Requirement origin is unresolved

• Approval authority is uncertain

• Threshold justification is incomplete

• Relevant operating conditions are unknown

• Requirement dependencies remain unverified

Uncertainty shall not be silently converted into requirement acceptance.

Unresolved Quality Requirement Conditions ≠ Established Quality

Requirement Conditions.

Quality Requirement Gap

A Quality Requirement Gap exists where material information required for competent requirement establishment remains insufficiently defined.

Examples may include:

• Missing intended-use context

• Undefined quality characteristic

• Incomplete requirement scope

• Missing authority evidence

• Unjustified threshold

• Unclear assessment expectation

QRM shall identify the gap and its material significance.

Quality Requirement Gap ≠ Proven Software Quality Failure.

Critical Quality Requirement Gap

A Critical Quality Requirement Gap exists where missing or unresolved requirement conditions prevent competent governance of a materially consequential software quality expectation.

Examples may include:

• A critical software function without defined reliability requirements

• A high-impact application without relevant recovery expectations

• A production system with materially undefined performance limits

• A software product whose intended use requires quality conditions that

have not been established.

QRM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software shutdown, release rejection, security compromise, or legal liability.

Critical Quality Requirement Gap ≠ Automatic Software Failure.

Quality Requirement Change

QRM shall recognize material changes to established quality requirements.

Changes may concern:

• Requirement scope

• Quality characteristic

• Threshold

• Priority

• Criticality

• Intended use

• Applicable environment

• Acceptance expectation

Change authority and relevant historical continuity shall remain explicit.

Broader lifecycle change management remains governed by SCMS.

Modified Quality Requirement ≠ Automatically Authorized Requirement

Change.

Quality Requirement Supersession

QRM shall preserve relationships between quality requirements that replace or revise earlier requirements.

Supersession may involve:

• Revised thresholds

• Updated intended-use conditions

• Changed platform requirements

• New reliability expectations

• Revised recovery conditions

Supersession shall not erase the historical validity or applicability of

earlier requirements.

Superseded Quality Requirement ≠ Historically Invalid Quality Requirement.

Quality Requirement Continuity

QRM shall preserve material requirement relationships across the software

lifecycle.

Continuity may involve:

• Original requirement references

• Approved revisions

• Changed software scope

• New operating conditions

• Updated quality expectations

• Downstream criteria

• Historical assessment references

Continuity shall preserve the distinction between historical requirements

and currently applicable requirements.

Historical Quality Requirement ≠ Currently Applicable Quality Requirement.

Quality Requirement Reassessment

QRM shall support reassessment when material quality requirement conditions change.

Triggers may include:

• Changed intended use

• New software versions

• Changed operating environment

• New stakeholder requirements

• New quality findings

• Revised technical constraints

• Changed dependencies

• Conflicting requirement evidence

Reassessment shall preserve relevant historical determinations.

Requirement Reassessment ≠ Historical Requirement Record Erasure.

Quality Requirement Governance States

QRM shall support identifiable governance states.

Possible states include:

Identified: A quality requirement governance object has been recognized.

Proposed: A quality expectation has been submitted for assessment.

Under Assessment: Material requirement conditions are being evaluated.

Specified: The requirement statement and relevant scope have been

established.

Justification-Pending: The requirement's supporting basis remains

incomplete.

Authority-Pending: Competent establishment authority remains unresolved.

Conditionally Established: The requirement is recognized subject to explicit conditions.

Governed: Material identity, scope, justification, authority, and

applicability conditions are sufficiently established.

Conflict-Detected: Material requirement claims are incompatible.

Evidence-Limited: Available evidence is insufficient for the required determination.

Critical-Gap: Missing material information prevents competent

establishment.

Change-Pending: A material requirement revision is under assessment.

Superseded: A later requirement has replaced the earlier reference.

Withdrawn: The requirement has been formally withdrawn by competent

authority.

Unresolved: Available information does not support a sufficiently determined state.

These states shall remain distinct from quality criteria satisfaction,

testing results, and quality acceptance.

Governed Quality Requirement ≠ Satisfied Quality Requirement.

Governed Software Quality Requirements State

The principal output of QRM is:

Governed Software Quality Requirements State

The state shall preserve sufficient information to identify:

Requirement Governance Object: The quality requirement assessment.

Requirement Identity: The identifiable requirement.

Requirement Purpose: The reason for establishment.

Software Object: The relevant software.

Intended Use: The applicable purpose and operating context.

Stakeholder Expectations: Material originating expectations.

Requirement Origin: The source of the requirement.

Requirement Justification: The supporting rationale.

Requirement Authority: The legitimate basis for establishment.

Requirement Scope: The applicable boundaries.

Quality Characteristic: The relevant quality dimension.

Requirement Classification: The recognized requirement type.

Requirement Statement: The expected quality condition.

Requirement Priority: The recognized importance.

Requirement Criticality: The material significance of non-satisfaction.

Quality Thresholds: Applicable measurable or qualitative expectations.

Requirement Constraints: Relevant restrictions.

Requirement Assumptions: Material underlying assumptions.

Requirement Dependencies: Relevant prerequisite relationships.

Requirement Consistency: The assessed relationship with other

requirements.

Requirement Completeness: The assessed coverage of material quality needs.

Requirement Feasibility: The recognized feasibility condition.

Requirement Assessability: The extent to which the requirement can be meaningfully evaluated.

Requirement Traceability: Relevant origin and downstream relationships.

Acceptance Expectations: Material quality acceptance conditions.

Requirement Evidence: Evidence supporting material claims.

Evidence Competence: The suitability of evidence for those claims.

Evidence Integrity: The condition of supporting evidence.

Requirement Conflicts: Contradictory requirement claims.

Requirement Uncertainty: Material unresolved conditions.

Requirement Gaps: Missing material requirement information.

Critical Requirement Gaps: Material gaps requiring competent attention.

Requirement Changes: Relevant revisions.

Requirement Supersession: Relationships to replaced requirements.

Requirement Continuity: Preservation of lifecycle relationships.

Requirement Reassessment: Material reassessment history.

Requirement Governance State: The recognized requirement condition.

Downstream References: Information required by QCDM and parent SQS

governance.

The state shall not independently establish quality criteria satisfaction,

test success, defect resolution, or software quality acceptance.

Minimum Implementation Framework

A conforming implementation of QRM must provide the following six functional capabilities.

• Quality Requirement Object, Purpose, Origin, Software Scope, and

Intended-Use Governance

Capability to establish identifiable quality requirement governance

objects, recognize stakeholder expectations, identify requirement purpose and origin, associate requirements with software objects, and establish applicable intended-use and scope boundaries.

The implementation must distinguish stakeholder quality expectations from governed quality requirements.

• Quality Characteristic, Classification, Priority, Criticality, and

Threshold Governance

Capability to identify relevant quality characteristics, classify quality

requirements, assess priority and criticality, and establish material qualitative or quantitative quality expectations.

The implementation must distinguish declared thresholds from justified requirement-level thresholds.

• Quality Requirement Authority, Justification, Constraints, Assumptions,

and Dependency Governance

Capability to preserve legitimate requirement establishment authority,

supporting justification, material constraints, assumptions, and prerequisite relationships.

The implementation must distinguish requirement origin and technical plausibility from competent authorization.

• Quality Requirement Consistency, Completeness, Feasibility,

Assessability, and Traceability Governance

Capability to assess whether material quality requirements are sufficiently coherent, complete within the defined scope, feasible, assessable, and traceable to relevant origins and downstream criteria.

The implementation must distinguish a documented requirement from an assessable and sufficiently governed requirement.

• Quality Requirement Evidence, Conflict, Uncertainty, Gap, Change, and

Continuity Governance

Capability to associate requirement claims with competent evidence,

preserve evidence integrity, identify conflicts, uncertainty, gaps, critical gaps, revisions, supersession, and lifecycle continuity.

The implementation must distinguish missing requirement evidence from demonstrated software quality failure.

• Governed Software Quality Requirements State and SQS Handoff Governance

Capability to produce a Governed Software Quality Requirements State with

sufficient identity, purpose, intended-use, scope, characteristic, justification, authority, threshold, assessability, evidence, uncertainty, and continuity information for competent downstream QCDM and parent SQS governance.

The implementation must not represent established quality requirements as proof that the software satisfies them.
GIE™ Compatibility

QRM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Quality requirement governance object identification

• Requirement purpose analysis

• Software object correspondence analysis

• Intended-use mapping

- Stakeholder expectation identification
- Requirement origin analysis
- Requirement justification assessment
- Requirement authority analysis
- Requirement scope assessment
- Quality characteristic identification
- Requirement classification
- Functional quality requirement analysis
- Reliability requirement analysis
- Performance requirement analysis
- Compatibility requirement analysis
- Usability requirement analysis
- Maintainability requirement analysis
- Portability requirement analysis
- Recoverability requirement analysis
- Requirement priority assessment
- Requirement criticality assessment
- Threshold analysis
- Constraint identification
- Assumption recognition
- Requirement dependency mapping
- Requirement consistency assessment
- Requirement completeness assessment
- Requirement feasibility analysis
- Requirement assessability assessment
- Requirement traceability mapping
- Quality acceptance expectation analysis
- Requirement evidence competence assessment
- Evidence integrity assessment
- Requirement conflict detection
- Requirement uncertainty recognition
- Requirement gap detection
- Critical requirement gap detection
- Requirement change analysis
- Requirement supersession mapping
- Requirement continuity mapping

• Requirement reassessment support

• Governed quality requirements state representation

GIE™ may identify candidate quality requirements, detect possible requirement conflicts, or propose quality requirement governance assessments.

Such outputs shall remain distinguishable from sufficiently established and

authorized requirements.

GIE™ must not fabricate requirement approvals, invent stakeholder expectations, infer authorization from technical feasibility, or represent proposed quality expectations as governed requirements without sufficient evidence.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical QRM governance logic is:

Receive Governed Software Requirements Baseline

↓

Receive Relevant Software Creation, Dependency, and Configuration States

↓

Identify Quality Requirement Governance Object

↓

Establish Requirement Purpose

↓

Identify Software Object and Intended Use

↓

Identify Stakeholder Quality Expectations

↓

Establish Requirement Origin

↓

Preserve Requirement Justification

↓

Identify Requirement Authority

↓

Establish Requirement Scope

↓

Identify Relevant Quality Characteristics

↓

Classify Quality Requirements

↓

Establish Requirement Statements

↓

Assess Priority and Criticality

↓

Identify Quality Thresholds

↓

Identify Constraints and Assumptions

↓

Map Requirement Dependencies

↓

Assess Requirement Consistency

↓

Assess Requirement Completeness

↓

Assess Requirement Feasibility

↓

Assess Requirement Assessability

↓

Establish Requirement Traceability

↓

Identify Quality Acceptance Expectations

↓

Associate Requirement Evidence

↓

Assess Evidence Competence and Integrity

↓

Identify Requirement Conflicts

↓

Identify Requirement Uncertainty

↓

Identify Requirement Gaps and Critical Gaps

↓

Determine Requirement Governance State

↓

Establish Governed Software Quality Requirements State

↓

Handoff to QCDM and Parent SQS Governance

↓

Preserve Requirement Change, Supersession, and Continuity

↓

Reassess When Material Requirement Conditions Change

The principal output is:

Governed Software Quality Requirements State

Governance Outputs

QRM may produce:

Quality Requirement Governance Object

Quality Requirement Identity Record

Quality Requirement Purpose Record

Software Object Correspondence Record

Intended-Use Correspondence Record

Stakeholder Quality Expectation Record

Quality Requirement Origin Record

Quality Requirement Justification Record

Quality Requirement Authority Record

Quality Requirement Scope Record

Quality Characteristic Register

Quality Requirement Classification Record

Functional Quality Requirement Record

Reliability Requirement Record

Performance Requirement Record

Compatibility Requirement Record

Usability Requirement Record

Maintainability Requirement Record

Portability Requirement Record

Recoverability Requirement Record

Quality Requirement Priority Assessment

Quality Requirement Criticality Assessment

Quality Requirement Threshold Record

Quality Requirement Constraint Record

Quality Requirement Assumption Record

Quality Requirement Dependency Record

Quality Requirement Consistency Assessment

Quality Requirement Completeness Assessment

Quality Requirement Feasibility Assessment

Quality Requirement Assessability Assessment

Quality Requirement Traceability Record

Quality Acceptance Expectation Record

Quality Requirement Evidence Register

Evidence Competence Assessment

Evidence Integrity Assessment

Quality Requirement Conflict

Quality Requirement Uncertainty State

Quality Requirement Gap

Critical Quality Requirement Gap

Quality Requirement Change Record

Quality Requirement Supersession Record

Quality Requirement Continuity Record

Quality Requirement Reassessment Record

Quality Requirement Governance State

SQS Quality Requirements Handoff Record

The principal output is:

Governed Software Quality Requirements State

Critical Distinctions

QRM preserves the following distinctions:

Quality Expectation ≠ Governed Quality Requirement

Quality Requirement Governance Object ≠ Stakeholder Quality Request

Quality Requirement Purpose ≠ Quality Requirement Satisfaction

Quality Requirement for One Software Object ≠ Quality Requirement for

Every Related Software Object

Quality Expectation in One Context ≠ Universal Quality Requirement

Stakeholder Quality Expectation ≠ Authorized Quality Requirement

Requirement Origin ≠ Requirement Authorization

Quality Requirement Justification ≠ Quality Requirement Approval

Ability to Propose a Quality Requirement ≠ Authority to Establish It

Defined Requirement Scope ≠ Universal Requirement Applicability

Quality Characteristic Identification ≠ Quality Characteristic

Demonstration

Quality Requirement Classification ≠ Quality Requirement Criticality

Defined Functional Quality Requirement ≠ Verified Functional Correctness

Reliability Requirement ≠ Demonstrated Reliability

Performance Target ≠ Demonstrated Performance

Compatibility Requirement for One Platform ≠ Universal Compatibility

Requirement

Usability Expectation ≠ Demonstrated Usability

Maintainability Requirement ≠ Demonstrated Maintainability

Portable Design Objective ≠ Verified Portability

Recovery Requirement ≠ Verified Recovery Capability

Requirement Priority ≠ Requirement Criticality

High Requirement Priority ≠ High Requirement Criticality

Declared Quality Threshold ≠ Justified Quality Threshold

Quality Requirement Constraint ≠ Quality Requirement Satisfaction

Quality Requirement Assumption ≠ Verified Operating Condition

Quality Requirement Dependency Identified ≠ Dependency Condition Satisfied

Individually Valid Requirements ≠ Collectively Consistent Requirements

Documented Quality Requirements ≠ Complete Quality Requirements

Feasible Quality Requirement ≠ Satisfied Quality Requirement

Quality Requirement Statement ≠ Assessable Quality Requirement

Traceable Quality Requirement ≠ Validated Quality Requirement

Quality Acceptance Expectation ≠ Quality Acceptance Decision

Quality Requirement Evidence Presence ≠ Requirement Validity

Evidence of Quality Need ≠ Evidence of Requirement Authorization

Intact Quality Requirement Record ≠ Governed Quality Requirement

Conflicting Quality Requirements ≠ Permission to Select the Preferred

Requirement

Unresolved Quality Requirement Conditions ≠ Established Quality

Requirement Conditions

Quality Requirement Gap ≠ Proven Software Quality Failure

Critical Quality Requirement Gap ≠ Automatic Software Failure

Modified Quality Requirement ≠ Automatically Authorized Requirement Change

Superseded Quality Requirement ≠ Historically Invalid Quality Requirement

Historical Quality Requirement ≠ Currently Applicable Quality Requirement

Requirement Reassessment ≠ Historical Requirement Record Erasure

Governed Quality Requirement ≠ Satisfied Quality Requirement

Cross-Architecture Boundary

QRM governs software-quality-specific requirement identity, purpose,

justification, authority, applicability, characteristics, thresholds, consistency, assessability, and continuity within SQS and SLGA™.

It does not independently govern the complete software requirements lifecycle, define detailed testing criteria, execute quality assessments, resolve defects, accept software quality, certify security, authorize release, or authorize deployment.

Within SQS:

QRM — Quality Requirements Module governs software quality requirement

establishment.

QCDM — Quality Criteria Definition Module governs the detailed

definition, applicability, and assessability of software quality criteria.

TCMM — Testing & Coverage Management Module governs test design,

execution, coverage, environment correspondence, and testing evidence.

DFM — Defect Management Module governs defect identification,

classification, impact, correction references, and resolution verification.

QCAM — Quality Conformity & Acceptance Module governs quality conformity

assessment, validation, acceptance authority, acceptance decisions, and residual limitations.

Within SLGA™:

Software Requirements Standard (SRS) governs the broader software

requirements baseline and lifecycle.

Software Creation Standard (SCS) governs software creation.

Software Dependency Standard (SDS) governs software dependencies.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs software retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

operational conditions relevant to quality requirements.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

requirements-related actions and boundaries.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions and

constituent action governance.

QRM preserves software-quality-requirement-specific governance without

replacing these competent domains.

Canonical Boundary

QRM begins when a material software quality expectation requires identification, justification, establishment, classification, authorization, revision, or reassessment.

QRM governs:

• Quality requirement governance objects

• Requirement identity

• Requirement purpose

• Software object correspondence

• Intended-use correspondence

• Stakeholder quality expectations

• Requirement origin

• Requirement justification

• Requirement authority

• Requirement scope

• Quality characteristic identification

• Requirement classification

• Functional quality requirements

• Reliability requirements

• Performance requirements

• Compatibility requirements

• Usability requirements

• Maintainability requirements

• Portability requirements

• Recoverability requirements

• Requirement priority

• Requirement criticality

• Requirement thresholds

• Requirement constraints

• Requirement assumptions

• Requirement dependencies

• Requirement consistency

• Requirement completeness

• Requirement feasibility

• Requirement assessability

• Requirement traceability

• Quality acceptance expectations

• Requirement evidence

• Evidence competence

• Evidence integrity

• Requirement conflicts

• Requirement uncertainty

• Requirement gaps

• Critical requirement gaps

• Requirement changes

• Requirement supersession

• Requirement continuity

• Requirement reassessment

• Requirement governance states

QRM does not independently determine:

• Complete software requirements validity

• Software creation authorization

• Dependency suitability

• Configuration baseline establishment

• Detailed quality criteria satisfaction

• Testing success

• Defect resolution

• Software quality acceptance

• Software security

• Release authorization

• Deployment authorization

• Legal compliance

• Legal liability

QRM ends when material software quality requirement conditions have been represented as a:

Governed Software Quality Requirements State

with sufficient requirement identity, purpose, intended-use, scope,

characteristic, justification, authority, threshold, assessability, evidence, uncertainty, and continuity information for competent QCDM and parent SQS governance.

Use Case 1 — A Software Team Defines Performance Requirements Without

an Operational Basis

An organization develops a cloud-based customer application.

The product team establishes a performance requirement:

"The application must respond within 100 milliseconds."

The requirement is included in the development backlog.

However, the requirement does not specify:

• Which operations must meet the threshold

• The expected number of concurrent users

• The relevant operating environment

• The measurement method

• Whether the threshold applies to average, median, or high-percentile

• The business or operational justification for the selected value

response time.

The development team treats the requirement as mandatory.

QRM evaluates:

Quality Expectation → Intended Use → Software Scope → Quality

Characteristic → Threshold → Justification → Authority → Assessability

The assessment identifies:

• A legitimate performance expectation

• An insufficiently defined requirement scope

• An unsupported numerical threshold

• Missing operational workload assumptions

• Incomplete assessability conditions

The relevant governance state may be:

Justification-Pending

or:

Evidence-Limited

until the requirement is sufficiently established.

The critical distinctions are:

Quality Expectation ≠ Governed Quality Requirement.

Declared Quality Threshold ≠ Justified Quality Threshold.

Quality Requirement Statement ≠ Assessable Quality Requirement.

The result: QRM prevents an arbitrary numerical target from becoming an

apparently authoritative software quality obligation without sufficient context, justification, and assessment boundaries.

Use Case 2 — An AI Agent Introduces a Quality Requirement Outside Its

Delegated Authority

An organization uses an autonomous development agent to improve the

maintainability of an internal application.

The agent is authorized to:

• Analyze source code

• Identify maintainability concerns

• Propose improvements

• Recommend quality criteria

During its assessment, the agent introduces a new mandatory requirement:

"All software components must be rewritten to satisfy a newly selected complexity threshold."

The agent modifies the development plan and begins restructuring several

components.

However, the agent was not authorized to establish new mandatory software

quality requirements or initiate large-scale architectural changes.

QRM evaluates:

Agent Objective → Quality Expectation → Requirement Origin → Authority →

Scope → Requirement Establishment → Downstream Actions

The assessment identifies:

• A legitimate maintainability improvement objective

• A proposed quality threshold

• No competent authorization establishing the threshold as mandatory

• An autonomous agent acting beyond its delegated requirements authority

• Downstream development actions initiated on the basis of an ungoverned

requirement.

The relevant governance state may be:

Authority-Pending

for the proposed requirement, while the unauthorized execution actions

remain subject to competent software creation and agent governance.

The critical distinctions are:

Ability to Propose a Quality Requirement ≠ Authority to Establish It.

Requirement Origin ≠ Requirement Authorization.

Quality Requirement Justification ≠ Quality Requirement Approval.

Task Authority ≠ Constituent Action Authority.

The result: QRM prevents an autonomous development system from converting

its own quality recommendations into binding requirements without legitimate establishment authority.

Canonical Closing Statement

Software quality requirements are not governed merely because they are written in a specification, requested by a stakeholder, generated by an AI system, or included in a development backlog.

A requirement may express a legitimate quality need while remaining insufficiently defined.

A performance threshold may appear precise while lacking an operational justification.

A reliability expectation may be important while remaining impossible to assess under its current formulation.

And an automated development agent may identify a valuable quality improvement without possessing authority to establish a new mandatory requirement.

Quality Expectation ≠ Governed Quality Requirement.

Stakeholder Quality Expectation ≠ Authorized Quality Requirement.

Declared Quality Threshold ≠ Justified Quality Threshold.

Quality Requirement Statement ≠ Assessable Quality Requirement.

Governed Quality Requirement ≠ Satisfied Quality Requirement.

These distinctions establish the foundation of competent software quality

governance.

Before software quality can be meaningfully tested, measured, validated, or accepted, the relevant quality requirements must be sufficiently established.

Their purpose must be identifiable.

Their applicability must be bounded.

Their justification must be explicit.

Their authority must be legitimate.

Their constraints and assumptions must remain visible.

And their relationship to subsequent quality criteria must remain traceable.

QRM does not determine whether software is good.

It determines whether the quality conditions against which software will be judged have been sufficiently established to support meaningful assessment.

QRM transforms software quality expectations from informal preferences,

technical aspirations, and unverified thresholds into governed requirements — ensuring that software organizations can establish not merely what quality they want, but why that quality is required, where it applies, who has authority to establish it, and how it can become a legitimate basis for subsequent quality governance.

A software quality requirement is not governed because it has been requested, documented, prioritized, or generated. It is governed when its identity, purpose, intended use, scope, justification, authority, applicability, assessability, evidence, and lifecycle continuity remain sufficiently established for competent software quality governance.